```python
import marimo

__generated_with = "0.25.0"
app = marimo.App(width="medium")


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    # 1. Introdução e definição do problema

    ## Descrição do problema

    O Game of Life é um autómato celular em que cada célula pode estar viva ou morta. O seu estado depende dos estados das células vizinhas.

    Neste trabalho considera-se uma grelha finita de dimensão \(N \times N\), com \(N > 16\). As células desta grelha são as células normais.

    Existem ainda duas categorias de células adicionais:
    - **Células iniciais:** têm um dos índices igual a zero. Os seus valores são gerados aleatoriamente no início, com probabilidade 1/2 de estarem vivas, e não mudam durante a execução.
    - **Células finais:** têm um dos índices igual a \(N+1\) e estão sempre mortas.

    As células normais seguem a regra B3/S23:
    - Uma célula morta nasce se tiver exatamente 3 vizinhos vivos.
    - Uma célula viva sobrevive se tiver 2 ou 3 vizinhos vivos.
    - Nos restantes casos fica ou passa a estar morta.

    Duas células são vizinhas quando a maior diferença absoluta entre as suas coordenadas é igual a 1. Assim, cada célula tem até 8 vizinhos, incluindo as diagonais.

    Inicialmente, as células normais estão todas mortas, exceto as nove células do canto superior esquerdo (região 3 × 3), que estão vivas. Considera-se estado de erro aquele em que todas as células normais estão mortas.

    ## Objetivos

    O objetivo é construir uma máquina de estados finita (FSM) que represente o sistema, considerando duas formas de transição:
    1. **Sequencial:** apenas uma célula normal muda de estado em cada transição.
    2. **Simultânea:** todas as células normais atualizam o seu estado ao mesmo tempo.

    As duas modalidades devem poder ser selecionadas através de um parâmetro no código.

    Pretende-se ainda verificar se o estado de erro pode ser alcançado e se existem células normais que permanecem sempre vivas ou sempre mortas.
    """)
    return


@app.cell
def _():
    import marimo as mo

    return (mo,)


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    # 2. Modelação e implementação

    A implementação foi dividida em duas partes. Primeiro, foi desenvolvido um modelo operacional em Python, que permite simular a evolução do autómato. Depois, foi construída uma representação simbólica em Z3, que permite verificar propriedades e procurar contraexemplos.

    ## 2.1. Simulação em Python

    Nesta primeira parte são definidas as estruturas que representam a grelha, as funções para consultar
```

```
os vizinhos e aplicar a regra B3/S23, e as duas modalidades de transição.
    """)
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Começamos por definir a grelha, distinguir os três tipos de células e construir o estado inicial,
atribuindo valores aleatórios às células iniciais adicionais.
    """)
    return


@app.cell
def _():

    import random

    def create_grid(N, seed=42):
        assert N > 16

        normal = {
            (i, j)
            for i in range(1, N + 1)
            for j in range(1, N + 1)
        }

        initial = (
            {(0, j) for j in range(N + 1)}
            | {(i, 0) for i in range(1, N + 1)}
        )

        final = (
            {(N + 1, j) for j in range(N + 2)}
            | {(i, N + 1) for i in range(N + 1)}
        )

        rng = random.Random(seed)

        boundary = {
            cell: rng.choice([False, True])
            for cell in initial
        }

        boundary.update({
            cell: False
            for cell in final
        })

        initial_state = {
            (i, j): (i <= 3 and j <= 3)
            for i, j in normal
        }

        return normal, initial, final, boundary, initial_state

    return (create_grid,)


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Testamos a construção da grelha para N = 17, verificando as quantidades de células de cada tipo e as
condições do estado inicial.
    """)
    return


@app.cell
```

```python
def _(create_grid):

    N = 17

    normal, initial, final, boundary, initial_state = create_grid(N)

    print("Células normais:", len(normal))
    print("Células iniciais:", len(initial))
    print("Células finais:", len(final))

    print("Estado inicial: células vivas:",
          sum(initial_state.values()))

    assert len(normal) == N**2
    assert len(initial) == 2*N + 1
    assert len(final) == 2*N + 3

    assert normal.isdisjoint(initial)
    assert normal.isdisjoint(final)
    assert initial.isdisjoint(final)

    assert sum(initial_state.values()) == 9

    print("Todos os testes passaram.")
    return boundary, final, initial, initial_state, normal


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Definimos as funções para obter os vizinhos de cada célula, consultar os seus valores e calcular o próximo estado segundo a regra B3/S23.
    """)
    return


@app.cell
def _():
    def neighbors(i, j):
        return [
            (i + di, j + dj)
            for di in (-1, 0, 1)
            for dj in (-1, 0, 1)
            if (di, dj) != (0, 0)
        ]


    def cell_value(state, boundary, cell):

        if cell in state:
            return int(state[cell])

        if cell in boundary:
            return int(boundary[cell])

        raise ValueError(f"Coordenada inválida: {cell}")


    def next_cell_value(state, boundary, i, j):

        alive_neighbors = sum(
            cell_value(state, boundary, cell)
            for cell in neighbors(i, j)
        )

        currently_alive = state[(i, j)]

        if currently_alive:
            return alive_neighbors in (2, 3)
```

```
        return alive_neighbors == 3

    return cell_value, neighbors, next_cell_value


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Criamos estados de teste e verificamos alguns casos da regra B3/S23.
    """)
    return


@app.cell
def _(next_cell_value):
    def empty_test_state():
        return {(i, j): False for i in range(1, 4)
                for j in range(1, 4)}


    def test_life_rule():
        boundary = {}

        # Uma célula morta com exatamente três vizinhos vivos nasce.
        s = empty_test_state()
        s[(1, 1)] = True
        s[(1, 2)] = True
        s[(2, 1)] = True
        assert next_cell_value(s, boundary, 2, 2)

        # Uma célula viva com dois vizinhos sobrevive.
        s = empty_test_state()
        s[(2, 2)] = True
        s[(1, 1)] = True
        s[(1, 2)] = True
        assert next_cell_value(s, boundary, 2, 2)

        # Uma célula viva com um vizinho morre.
        s = empty_test_state()
        s[(2, 2)] = True
        s[(1, 1)] = True
        assert not next_cell_value(s, boundary, 2, 2)

        # Uma célula viva com quatro vizinhos morre.
        s = empty_test_state()
        s[(2, 2)] = True
        for cell in [(1, 1), (1, 2), (1, 3), (2, 1)]:
            s[cell] = True
        assert not next_cell_value(s, boundary, 2, 2)

        print("Todos os testes B3/S23 passaram.")


    test_life_rule()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Implementamos as duas modalidades de evolução do autómato: a atualização simultânea de todas as
células e a atualização sequencial de uma única célula de cada vez.
    """)
    return


@app.cell
def _(next_cell_value):
    def simultaneous_step(state, boundary):
```

```
        return {
            cell: next_cell_value(state, boundary, *cell)
            for cell in state
        }


    def sequential_successors(state, boundary):

        successors = []

        for cell in state:
            new_value = next_cell_value(state, boundary, *cell)

            if new_value != state[cell]:
                new_state = state.copy()
                new_state[cell] = new_value
                successors.append(new_state)

        return successors

    return sequential_successors, simultaneous_step


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Testamos as duas modalidades para verificar se os sucessores produzidos respeitam as regras de
transição.
    """)
    return


@app.cell
def _(
    boundary,
    initial_state,
    next_cell_value,
    normal,
    sequential_successors,
    simultaneous_step,
):

    def test_transition_modes(normal_cells, boundary_values, initial_configuration):

        # Modalidade simultânea
        simultaneous_result = simultaneous_step(
            initial_configuration, boundary_values
        )

        assert set(simultaneous_result) == set(normal_cells)
        assert set(simultaneous_result) == set(initial_configuration)

        # Modalidade sequencial
        seq_successors = sequential_successors(
            initial_configuration, boundary_values
        )

        assert len(seq_successors) > 0

        for successor in seq_successors:

            differences = [
                cell for cell in normal_cells
                if successor[cell] != initial_configuration[cell]
            ]

            assert len(differences) == 1

            changed_cell = differences[0]
```

```
            assert successor[changed_cell] == next_cell_value(
                initial_configuration, boundary_values, *changed_cell
            )

            for cell in normal_cells - {changed_cell}:
                assert successor[cell] == initial_configuration[cell]

        print("Transição simultânea: OK")
        print("Transição sequencial: OK")
        print("Sucessores sequenciais:", len(seq_successors))

        return len(seq_successors)


    test_transition_modes(normal, boundary, initial_state)
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    ## 2.2. Modelação simbólica em Z3

    Nesta parte representamos os estados e as transições através de variáveis e restrições booleanas, permitindo utilizar o solver Z3 para verificar propriedades do autómato.
    """)
    return


@app.cell
def _(initial, normal):

    import z3

    # Todas as células cujo valor faz parte do estado.
    # As células finais são constantes iguais a False.
    symbolic_cells = tuple(sorted(normal | initial))

    def create_symbolic_state(prefix, cells):
        return {
            cell: z3.Bool(f"{prefix}_{cell[0]}_{cell[1]}")
            for cell in cells
        }


    # Dois estados distintos: antes e depois de uma transição.
    X = create_symbolic_state("x", symbolic_cells)
    Y = create_symbolic_state("y", symbolic_cells)


    print("Variáveis Booleanas por estado:", len(X))
    print("Variáveis de estado:", len(X))
    print("Variáveis do estado seguinte:", len(Y))
    return X, Y, create_symbolic_state, symbolic_cells, z3


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Definimos o predicado que caracteriza os estados iniciais, fixando as nove células normais inicialmente vivas.
    """)
    return


@app.cell
def _(X, normal, z3):

    def initial_predicate(X, normal_cells):
        return z3.And(*[
```

```
            X[cell] == z3.BoolVal(
                cell[0] <= 3 and cell[1] <= 3
            )
            for cell in normal_cells
        ])


    I = initial_predicate(X, normal)

    print("Predicado inicial construído.")
    return I, initial_predicate


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Definimos o predicado que identifica os estados de erro, nos quais todas as células normais estão
mortas.
    """)
    return


@app.cell
def _(X, normal, z3):

    def error_predicate(X, normal_cells):
        return z3.And(*[
            z3.Not(X[cell])
            for cell in normal_cells
        ])


    E = error_predicate(X, normal)

    print("Predicado de erro construído.")
    return E, error_predicate


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Traduzimos a regra B3/S23 para expressões booleanas em Z3, permitindo calcular simbolicamente o
próximo estado de cada célula.
    """)
    return


@app.cell
def _(neighbors, z3):

    def z3_next_cell(state, cell):

        alive_neighbors = [
            state.get(neighbor, z3.BoolVal(False))
            for neighbor in neighbors(*cell)
        ]

        count = z3.Sum([
            z3.If(value, 1, 0)
            for value in alive_neighbors
        ])

        alive = state[cell]

        return z3.If(
            alive,
            z3.Or(count == 2, count == 3),
            count == 3
        )
```

```
    return (z3_next_cell,)


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Definimos a relação de transição da FSM, contemplando os modos sequencial e simultâneo.
    """)
    return


@app.cell
def _(X, Y, initial, normal, z3, z3_next_cell):

    def transition_predicate(
        X,
        Y,
        normal_cells,
        initial_cells,
        mode,
        selector_prefix="selected"
    ):
        constraints = []

        # As células iniciais adicionais mantêm sempre o seu valor.
        for cell in initial_cells:
            constraints.append(Y[cell] == X[cell])

        if mode == "simultaneous":

            # Todas as células normais evoluem em simultâneo.
            for cell in normal_cells:
                constraints.append(
                    Y[cell] == z3_next_cell(X, cell)
                )

        elif mode == "sequential":

            # Uma variável auxiliar indica qual a célula escolhida.
            selected = {
                cell: z3.Bool(
                    f"{selector_prefix}_{cell[0]}_{cell[1]}"
                )
                for cell in normal_cells
            }

            # Exatamente uma célula é selecionada.
            constraints.append(
                z3.PbEq(
                    [(selected[cell], 1) for cell in normal_cells],
                    1
                )
            )

            for cell in normal_cells:

                rule = z3_next_cell(X, cell)

                # A célula selecionada tem de mudar efetivamente.
                constraints.append(
                    z3.Implies(
                        selected[cell],
                        z3.And(
                            Y[cell] == rule,
                            Y[cell] != X[cell]
                        )
                    )
                )
```

```
                # Todas as restantes células permanecem iguais.
                constraints.append(
                    z3.Implies(
                        z3.Not(selected[cell]),
                        Y[cell] == X[cell]
                    )
                )

        else:
            raise ValueError(
            )

        return z3.And(*constraints)


    T_sim = transition_predicate(
        X, Y, normal, initial,
        mode="simultaneous"
    )

    T_seq = transition_predicate(
        X, Y, normal, initial,
        mode="sequential"
    )

    print("Relação simultânea construída.")
    print("Relação sequencial construída.")
    return T_seq, T_sim, transition_predicate


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    # 3. Verificação das propriedades e análise dos resultados

    Começamos por verificar algumas condições básicas da FSM, nomeadamente a validade do estado inicial e a existência de transições possíveis.
    """)
    return


@app.cell
def _(E, I, T_seq, T_sim, z3):

    def check_initial_transition(I, T):
        solver = z3.Solver()
        solver.add(I, T)
        return solver.check()


    # O estado inicial não pode ser um estado de erro.
    solver_error = z3.Solver()
    solver_error.add(I, E)

    print("Estado inicial é válido:", check_initial_transition(
        I, z3.BoolVal(True)
    ))
    print("Erro no estado inicial:", solver_error.check())

    # Tem de existir pelo menos uma transição possível.
    print("Transição simultânea:", check_initial_transition(I, T_sim))
    print("Transição sequencial:", check_initial_transition(I, T_seq))
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Construímos uma pesquisa limitada a um número de transições para verificar se é possível alcançar o estado de erro em cada modalidade.
```

```
    """)
    return


@app.cell
def _(
    create_symbolic_state,
    error_predicate,
    initial,
    initial_predicate,
    normal,
    symbolic_cells,
    transition_predicate,
    z3,
):

    def bounded_error_query(k, mode, all_initial_dead=True):

        # Criar k+1 estados simbolicos.
        states = [
            create_symbolic_state(
                f"bmc_{mode}_{k}_{t}",
                symbolic_cells
            )
            for t in range(k + 1)
        ]

        constraints = [
            initial_predicate(states[0], normal)
        ]

        # Fixar as celulas iniciais adicionais a zero.
        if all_initial_dead:
            constraints.extend([
                z3.Not(states[0][cell])
                for cell in initial
            ])

        # Construir a sequencia de k transicoes.
        for t in range(k):
            constraints.append(
                transition_predicate(
                    states[t],
                    states[t + 1],
                    normal,
                    initial,
                    mode,
                    selector_prefix=f"bmc_{mode}_{k}_step_{t}"
                )
            )

        # Exigir que o ultimo estado seja de erro.
        constraints.append(
            error_predicate(states[k], normal)
        )

        return z3.And(*constraints), states


    # Procurar um erro em exatamente 9 transicoes sequenciais.
    query_seq_9, states_seq_9 = bounded_error_query(
        k=9,
        mode="sequential",
        all_initial_dead=True
    )

    solver_seq_9 = z3.Solver()
    solver_seq_9.set("timeout", 120000)
    solver_seq_9.add(query_seq_9)
```

```
    result_seq_9 = solver_seq_9.check()

    print("BMC sequencial, 9 transicoes:", result_seq_9)

    if result_seq_9 == z3.sat:

        model_seq_9 = solver_seq_9.model()

        print("\nContraexemplo encontrado:")

        for t in range(9):

            changes = [
                cell for cell in normal
                if z3.is_true(model_seq_9.eval(
                    states_seq_9[t][cell],
                    model_completion=True
                )) != z3.is_true(model_seq_9.eval(
                    states_seq_9[t+1][cell],
                    model_completion=True
                ))
            ]

            print(f"Passo {t+1}: {changes}")

        print("\nEstado final de erro confirmado.")

    elif result_seq_9 == z3.unknown:
        print("O solver nao conseguiu concluir dentro do limite.")
    else:
        print("Nao foi encontrado erro neste limite.")
    return bounded_error_query, query_seq_9, states_seq_9


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Reproduzimos em Python a sequência de transições sequenciais que permite alcançar o estado de erro, verificando cada atualização.
    """)
    return


@app.cell
def _(cell_value, final, initial, neighbors, next_cell_value, normal):

    def independently_check_counterexample():

        dead_boundary = {
            cell: False for cell in initial | final
        }

        state = {
            cell: (cell[0] <= 3 and cell[1] <= 3)
            for cell in normal
        }

        moves = [
            (2, 2), (1, 2), (3, 2),
            (1, 3), (2, 3), (1, 1),
            (3, 3), (2, 1), (3, 1)
        ]

        for step, cell in enumerate(moves, start=1):

            assert state[cell] is True

            count = sum(
                cell_value(state, dead_boundary, neighbor)
                for neighbor in neighbors(*cell)
```

```
            )

            new_value = next_cell_value(
                state, dead_boundary, *cell
            )

            assert new_value is False

            state[cell] = new_value

            print(
                f"Passo {step}: {cell}, "
                f"{count} vizinhos vivos"
            )

        assert not any(state.values())

        print("\nContraexemplo validado independentemente!")
        print("Estado final de erro confirmado.")


    independently_check_counterexample()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Testamos a evolução simultânea com todas as células iniciais adicionais mortas e verificamos se o sistema chega a um estado estável.
    """)
    return


@app.cell
def _(final, initial, normal, simultaneous_step):

    def test_simultaneous_dead_boundary():

        dead_boundary = {
            cell: False for cell in initial | final
        }

        state = {
            cell: (cell[0] <= 3 and cell[1] <= 3)
            for cell in normal
        }

        previous = None

        for generation in range(10):

            alive = sum(state.values())

            print(
                f"Geração {generation}: "
                f"{alive} células vivas"
            )

            if previous == state:
                print("Estado estável encontrado.")
                break

            previous = state.copy()

            state = simultaneous_step(
                state,
                dead_boundary
            )
```

```
        assert sum(state.values()) == 7
        assert state == previous

        return state


    stable_state = test_simultaneous_dead_boundary()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Procuramos, através de Z3, uma trajetória simultânea que permita alcançar o estado de erro, considerando diferentes configurações das células iniciais adicionais.
    """)
    return


@app.cell
def _(bounded_error_query, initial, normal, z3):

    def search_simultaneous_error(max_steps=8):

        for k in range(1, max_steps + 1):

            query, states = bounded_error_query(
                k=k,
                mode="simultaneous",
                all_initial_dead=False
            )

            solver = z3.Solver()
            solver.set("timeout", 30000)
            solver.add(query)

            result = solver.check()

            print(f"Passos: {k} | Resultado: {result}")

            if result == z3.sat:

                model = solver.model()

                print("\nContraexemplo encontrado!")

                live_boundary = [
                    cell for cell in initial
                    if z3.is_true(model.eval(
                        states[0][cell],
                        model_completion=True
                    ))
                ]

                print("Células iniciais vivas:", live_boundary)

                for t, state in enumerate(states):
                    count = sum(
                        z3.is_true(model.eval(
                            state[cell],
                            model_completion=True
                        ))
                        for cell in normal
                    )
                    print(f"Geração {t}: {count} células vivas")

                return "sat"

            if result == z3.unknown:
                print("Limite de execução atingido ou resultado desconhecido.")
```

```
                print("Motivo:", solver.reason_unknown())
                return "unknown"

        print(
            f"\nNão foi encontrado estado de erro "
            f"até {max_steps} transições."
        )

        return "bounded_unsat"


    simultaneous_result = search_simultaneous_error()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Reproduzimos em Python o contraexemplo simultâneo encontrado por Z3 e confirmamos a evolução do
número de células vivas até ao estado de erro.
    """)
    return


@app.cell
def _(final, initial, normal, simultaneous_step):

    def independently_check_simultaneous():

        # Configuração devolvida pelo Z3
        live_initial = {
            (0, 16), (0, 13), (7, 0),
            (8, 0), (0, 2), (0, 11),
            (0, 17), (2, 0), (13, 0)
        }

        boundary_values = {
            cell: cell in live_initial
            for cell in initial
        }

        boundary_values.update({
            cell: False for cell in final
        })

        state = {
            cell: cell[0] <= 3 and cell[1] <= 3
            for cell in normal
        }

        expected_counts = [9, 3, 1, 0]

        for generation, expected in enumerate(expected_counts):

            alive = sum(state.values())

            print(
                f"Geração {generation}: {alive} células vivas"
            )

            assert alive == expected

            if generation < 3:
                state = simultaneous_step(
                    state, boundary_values
                )

        # Confirmar que o estado de erro é permanente.
        next_state = simultaneous_step(
            state, boundary_values
```

```
        )

        assert state == next_state
        assert not any(state.values())

        print("\nContraexemplo simultâneo validado!")
        print("O estado de erro é também um ponto fixo.")


    independently_check_simultaneous()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Definimos condições para identificar estados em que nenhuma célula normal pode mudar e introduzimos uma transição de permanência (stuttering) para representar a continuação de uma execução nesses estados.
    """)
    return


@app.cell
def _(symbolic_cells, transition_predicate, z3, z3_next_cell):

    def deadlock_predicate(state, normal_cells):
        return z3.And(*[
            state[cell] == z3_next_cell(state, cell)
            for cell in normal_cells
        ])


    def sequential_trace_transition(X, Y, normal_cells, initial_cells):

        ordinary_transition = transition_predicate(
            X, Y, normal_cells, initial_cells,
            mode="sequential",
            selector_prefix="trace_selected"
        )

        terminal_stutter = z3.And(
            deadlock_predicate(X, normal_cells),
            *[
                Y[cell] == X[cell]
                for cell in symbolic_cells
            ]
        )

        return z3.Or(
            ordinary_transition,
            terminal_stutter
        )


    return deadlock_predicate, sequential_trace_transition


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Verificamos se o estado de erro encontrado no modo sequencial é um estado terminal, ou seja, se deixa de ser possível realizar atualizações sequenciais.
    """)
    return


@app.cell
def _(deadlock_predicate, normal, query_seq_9, states_seq_9, z3):

    terminal_solver = z3.Solver()
```

```
    terminal_solver.add(
        query_seq_9,
        deadlock_predicate(states_seq_9[-1], normal)
    )

    terminal_result = terminal_solver.check()

    print("Estado final é terminal:", terminal_result)

    if terminal_result == z3.sat:
        print("Todas as células normais estão mortas.")
        print("Nenhuma atualização sequencial é possível.")
        print("O traço repete o estado final indefinidamente.")
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Verificamos se o estado de erro alcançado no modo simultâneo pode permanecer inalterado numa transição seguinte.
    """)
    return


@app.cell
def _(
    bounded_error_query,
    create_symbolic_state,
    initial,
    normal,
    symbolic_cells,
    transition_predicate,
    z3,
):

    query_sim_fixed, states_sim_fixed = bounded_error_query(
        k=3,
        mode="simultaneous",
        all_initial_dead=False
    )

    state_after_error = create_symbolic_state(
        "sim_after_error",
        symbolic_cells
    )

    final_transition = transition_predicate(
        states_sim_fixed[-1],
        state_after_error,
        normal,
        initial,
        mode="simultaneous"
    )

    same_state = z3.And(*[
        states_sim_fixed[-1][cell] == state_after_error[cell]
        for cell in symbolic_cells
    ])

    fixed_point_solver = z3.Solver()

    fixed_point_solver.set("timeout", 120000)

    fixed_point_solver.add(
        query_sim_fixed,
        final_transition,
        same_state
    )
```

```
    fixed_point_result = fixed_point_solver.check()

    print("Erro seguido de ponto fixo:", fixed_point_result)

    if fixed_point_result == z3.sat:
        print("Existe uma trajetória que chega ao erro")
        print("e permanece nesse estado indefinidamente.")
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Testamos a trajetória sequencial até ao estado de erro e verificamos que a introdução da transição de permanência permite prolongar o traço.
    """)
    return


@app.cell
def _(final, initial, next_cell_value, normal):

    def test_sequential_trace_semantics():

        # Usamos a configuração inicial com fronteiras mortas.
        dead_boundary = {
            cell: False for cell in initial | final
        }

        state = {
            cell: cell[0] <= 3 and cell[1] <= 3
            for cell in normal
        }

        # Executar o contraexemplo conhecido.
        moves = [
            (2, 2), (1, 2), (3, 2),
            (1, 3), (2, 3), (1, 1),
            (3, 3), (2, 1), (3, 1)
        ]

        for cell in moves:
            assert next_cell_value(
                state, dead_boundary, *cell
            ) != state[cell]

            state[cell] = next_cell_value(
                state, dead_boundary, *cell
            )

        # O estado final tem todas as células normais mortas.
        assert not any(state.values())

        # Nenhuma célula pode mudar.
        assert all(
            next_cell_value(state, dead_boundary, *cell)
            == state[cell]
            for cell in normal
        )

        # A extensão terminal mantém o estado.
        next_state = state.copy()

        assert next_state == state

        print("Atualizações sequenciais: OK")
        print("Estado terminal: OK")
        print("Transição de permanência: OK")
```

```
    test_sequential_trace_semantics()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Definimos uma função auxiliar para verificar se um estado está totalmente morto, é estável no modo simultâneo ou constitui um bloqueio no modo sequencial.
    """)
    return


@app.cell
def _(sequential_successors, simultaneous_step):

    def verify_permanent_dead_state(state, boundary):

        # Todas as células normais estão mortas.
        all_dead = not any(state.values())

        # O próximo estado simultâneo é idêntico.
        simultaneous_fixed = (
            simultaneous_step(state, boundary) == state
        )

        # Não existe nenhuma atualização sequencial possível.
        sequential_deadlock = (
            len(sequential_successors(state, boundary)) == 0
        )

        return {
            "all_dead": all_dead,
            "simultaneous_fixed": simultaneous_fixed,
            "sequential_deadlock": sequential_deadlock
        }


    return (verify_permanent_dead_state,)


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Aplicamos a função anterior aos estados de erro encontrados nas duas modalidades de transição.
    """)
    return


@app.cell
def _(final, initial, normal, verify_permanent_dead_state):

    # Primeiro contraexemplo: fronteiras todas mortas.
    dead_boundary = {
        cell: False for cell in initial | final
    }

    sequential_error_state = {
        cell: False for cell in normal
    }

    print(
        verify_permanent_dead_state(
            sequential_error_state,
            dead_boundary
        )
    )

    # Segundo contraexemplo: fronteiras da solução Z3.
```

```
    live_initial = {
        (0, 16), (0, 13), (7, 0),
        (8, 0), (0, 2), (0, 11),
        (0, 17), (2, 0), (13, 0)
    }

    sim_boundary = {
        cell: cell in live_initial
        for cell in initial
    }

    sim_boundary.update({
        cell: False for cell in final
    })

    simultaneous_error_state = {
        cell: False for cell in normal
    }

    print(
        verify_permanent_dead_state(
            simultaneous_error_state,
            sim_boundary
        )
    )
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Criamos uma verificação simbólica em Z3 para determinar se um estado pode transitar normalmente para
si próprio em cada modalidade.
    """)
    return


@app.cell
def _(create_symbolic_state, symbolic_cells, transition_predicate, z3):
    def check_terminal_self_loop(state, boundary, normal_cells,
                                 initial_cells, mode):

        X_terminal = create_symbolic_state(
            "terminal_X", symbolic_cells
        )
        Y_terminal = create_symbolic_state(
            "terminal_Y", symbolic_cells
        )

        constraints = []

        # Fixar as células normais no estado que queremos testar.
        for cell in normal_cells:
            constraints.append(
                X_terminal[cell] == z3.BoolVal(state[cell])
            )

        # Fixar os valores das células iniciais adicionais.
        for cell in initial_cells:
            constraints.append(
                X_terminal[cell] == z3.BoolVal(boundary[cell])
            )

        # Exigir que o estado seguinte seja igual ao estado atual.
        for cell in symbolic_cells:
            constraints.append(
                Y_terminal[cell] == X_terminal[cell]
            )

        solver = z3.Solver()
```

```
        solver.add(*constraints)

        # Verificar a existência de uma transição segundo o modo escolhido.
        solver.add(
            transition_predicate(
                X_terminal,
                Y_terminal,
                normal_cells,
                initial_cells,
                mode=mode,
                selector_prefix=f"terminal_{mode}"
            )
        )

        return solver.check() == z3.sat

    return (check_terminal_self_loop,)


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Testamos a existência de uma transição do estado totalmente morto para si próprio nos modos sequencial e simultâneo.
    """)
    return


@app.cell
def _(check_terminal_self_loop, initial, normal):
    def test_terminal_states():
        all_dead_state = {
            cell: False for cell in normal
        }

        all_dead_boundary = {
            cell: False for cell in initial
        }

        return {
            "Sequencial": check_terminal_self_loop(
                all_dead_state,
                all_dead_boundary,
                normal,
                initial,
                "sequential"
            ),
            "Simultâneo": check_terminal_self_loop(
                all_dead_state,
                all_dead_boundary,
                normal,
                initial,
                "simultaneous"
            )
        }

    test_terminal_states()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Confirmamos se o estado de erro do contraexemplo simultâneo também permite uma transição para si próprio.
    """)
    return


@app.cell
```

```
def _(check_terminal_self_loop, initial, normal):
    def test_simultaneous_counterexample_terminal():

        live_initial_cells = {
            (0,16), (0,13), (7,0), (8,0), (0,2),
            (0,11), (0,17), (2,0), (13,0)
        }

        sim_boundary = {
            cell: cell in live_initial_cells
            for cell in initial
        }

        all_dead_state = {
            cell: False for cell in normal
        }

        return check_terminal_self_loop(
            all_dead_state,
            sim_boundary,
            normal,
            initial,
            "simultaneous"
        )


    test_simultaneous_counterexample_terminal()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Reproduzimos os contraexemplos encontrados nos dois modos e registamos a evolução do número de
células vivas até ao estado de erro.
    """)
    return


@app.cell
def _(final, initial, initial_state, next_cell_value, simultaneous_step):
    def reproduce_both_counterexamples():

        # 1. Modo sequencial
        sequential_boundary = {
            cell: False for cell in initial | final
        }

        sequential_order = [
            (2,2), (1,2), (3,2),
            (1,3), (2,3), (1,1),
            (3,3), (2,1), (3,1)
        ]

        sequential_state = initial_state.copy()
        sequential_counts = [sum(sequential_state.values())]

        for cell in sequential_order:
            sequential_state[cell] = next_cell_value(
                sequential_state,
                sequential_boundary,
                *cell
            )
            sequential_counts.append(sum(sequential_state.values()))

        # 2. Modo simultâneo
        live_initial_cells = {
            (0,16), (0,13), (7,0), (8,0), (0,2),
            (0,11), (0,17), (2,0), (13,0)
        }
```

```
        simultaneous_boundary = {
            cell: cell in live_initial_cells
            for cell in initial
        }
        simultaneous_boundary.update({
            cell: False for cell in final
        })

        simultaneous_state = initial_state.copy()
        simultaneous_counts = [sum(simultaneous_state.values())]

        for _ in range(3):
            simultaneous_state = simultaneous_step(
                simultaneous_state,
                simultaneous_boundary
            )
            simultaneous_counts.append(sum(simultaneous_state.values()))

        return {
            "Sequencial - células vivas por estado": sequential_counts,
            "Sequencial - todas mortas": not any(sequential_state.values()),
            "Simultâneo - células vivas por geração": simultaneous_counts,
            "Simultâneo - todas mortas": not any(simultaneous_state.values())
        }


    reproduce_both_counterexamples()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    ### Testes de eficiência e escalabilidade

    Medimos o tempo de execução da simulação em Python para diferentes dimensões da grelha, considerando
20 gerações em cada caso.
    """)
    return


@app.cell
def _(create_grid, simultaneous_step):
    import time

    def benchmark_simulation_sizes():
        results = []

        for size in [17, 30, 50, 100, 150]:
            normal_cells, initial_cells, final_cells, boundary_cells, state = (
                create_grid(size)
            )

            # Incluir as células finais, sempre mortas
            boundary_cells.update({
                cell: False for cell in final_cells
            })

            start = time.perf_counter()

            # Simular 20 gerações
            for _ in range(20):
                state = simultaneous_step(state, boundary_cells)

            elapsed = time.perf_counter() - start

            results.append({
                "N": size,
                "Células normais": size ** 2,
```

```
                "Gerações": 20,
                "Tempo (segundos)": round(elapsed, 4)
            })

        return results


    benchmark_simulation_sizes()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Avaliamos o desempenho do Z3 na procura de uma trajetória simultânea até ao estado de erro, utilizando grelhas de diferentes dimensões.
    """)
    return


@app.cell
def _(
    create_symbolic_state,
    error_predicate,
    initial_predicate,
    transition_predicate,
    z3,
):
    def benchmark_z3_simultaneous():
        import time as benchmark_clock

        results = []

        for size in [17, 20, 25, 30]:
            normal_cells = {
                (i, j)
                for i in range(1, size + 1)
                for j in range(1, size + 1)
            }

            initial_cells = (
                {(0, j) for j in range(size + 1)}
                | {(i, 0) for i in range(1, size + 1)}
            )

            symbolic_cells_local = tuple(
                sorted(normal_cells | initial_cells)
            )

            states = [
                create_symbolic_state(
                    f"bench_{size}_{t}",
                    symbolic_cells_local
                )
                for t in range(4)
            ]

            solver = z3.Solver()
            solver.set(timeout=10000)

            solver.add(initial_predicate(states[0], normal_cells))

            for t in range(3):
                solver.add(
                    transition_predicate(
                        states[t],
                        states[t + 1],
                        normal_cells,
                        initial_cells,
                        "simultaneous",
```

```
                        selector_prefix=f"bench_{size}_{t}"
                    )
                )

            solver.add(error_predicate(states[3], normal_cells))

            start = benchmark_clock.perf_counter()
            answer = solver.check()
            elapsed = benchmark_clock.perf_counter() - start

            results.append({
                "N": size,
                "Células normais": size ** 2,
                "Resultado Z3": str(answer),
                "Tempo (segundos)": round(elapsed, 4)
            })

        return results


    benchmark_z3_simultaneous()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Verificamos simbolicamente em Z3 a validade da sequência de nove transições sequenciais que conduz ao estado de erro.
    """)
    return


@app.cell
def _(
    create_symbolic_state,
    error_predicate,
    initial_predicate,
    transition_predicate,
    z3,
    z3_next_cell,
):
    def verify_known_sequential_witness():
        size = 17

        normal_cells = {
            (i, j)
            for i in range(1, size + 1)
            for j in range(1, size + 1)
        }

        initial_cells = (
            {(0, j) for j in range(size + 1)}
            | {(i, 0) for i in range(1, size + 1)}
        )

        cells = tuple(sorted(normal_cells | initial_cells))

        states = [
            create_symbolic_state(f"witness_{t}", cells)
            for t in range(10)
        ]

        sequence = [
            (2,2), (1,2), (3,2),
            (1,3), (2,3), (1,1),
            (3,3), (2,1), (3,1)
        ]

        solver = z3.Solver()
```

```
        solver.add(initial_predicate(states[0], normal_cells))

        for cell in initial_cells:
            solver.add(z3.Not(states[0][cell]))

        for t, selected_cell in enumerate(sequence):
            solver.add(
                transition_predicate(
                    states[t],
                    states[t + 1],
                    normal_cells,
                    initial_cells,
                    "sequential",
                    selector_prefix=f"witness_selected_{t}"
                )
            )

            for cell in normal_cells:
                if cell == selected_cell:
                    solver.add(
                        states[t + 1][cell] ==
                        z3_next_cell(states[t], cell)
                    )
                else:
                    solver.add(
                        states[t + 1][cell] == states[t][cell]
                    )

        solver.add(error_predicate(states[9], normal_cells))

        return solver.check()


    verify_known_sequential_witness()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Medimos o desempenho do Z3 na pesquisa de trajetórias sequenciais até ao estado de erro, para diferentes dimensões da grelha, considerando nove transições.
    """)
    return


@app.cell
def _(
    create_grid,
    create_symbolic_state,
    error_predicate,
    initial_predicate,
    transition_predicate,
    z3,
):
    def benchmark_sequential_one_size(size, horizon=9, timeout_ms=30000):
        import time

        normal_cells, initial_cells, final_cells, boundary_values, initial_state = create_grid(size)

        cells = tuple(sorted(normal_cells | initial_cells))

        states = [
            create_symbolic_state(f"bench_{size}_{step}", cells)
            for step in range(horizon + 1)
        ]

        solver = z3.Solver()
        solver.set(timeout=timeout_ms)
```

```
        solver.add(initial_predicate(states[0], normal_cells))

        # Consideramos todas as células iniciais adicionais mortas.
        for cell in initial_cells:
            solver.add(states[0][cell] == z3.BoolVal(False))

        for step in range(horizon):
            solver.add(
                transition_predicate(
                    states[step],
                    states[step + 1],
                    normal_cells,
                    initial_cells,
                    mode="sequential",
                    selector_prefix=f"bench_{size}_step_{step}"
                )
            )

        solver.add(error_predicate(states[horizon], normal_cells))

        start = time.perf_counter()
        result = solver.check()
        elapsed = time.perf_counter() - start

        return {
            "N": size,
            "Células normais": len(normal_cells),
            "Transições": horizon,
            "Resultado": str(result),
            "Tempo (s)": round(elapsed, 4),
            "Razão": solver.reason_unknown() if result == z3.unknown else "-"
        }


    benchmark_sequential_one_size(17)
    return (benchmark_sequential_one_size,)


@app.cell
def _(benchmark_sequential_one_size):
    benchmark_sequential_one_size(20)
    return


@app.cell
def _(benchmark_sequential_one_size):
    benchmark_sequential_one_size(25)
    return


@app.cell
def _(benchmark_sequential_one_size):
    benchmark_sequential_one_size(30)
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Analisamos uma célula normal que permanece morta ao longo dos contraexemplos encontrados, nos dois
modos de transição.
    """)
    return


@app.cell
def _(create_grid, next_cell_value, simultaneous_step):
    def demonstrate_permanently_dead_cell():
        # Célula que vamos acompanhar nos dois modos.
```

```
target = (10, 10)

# ------------------------------------------------
# 1. Modo sequencial
# ------------------------------------------------
normal_seq, initial_seq, final_seq, boundary_seq, state_seq = create_grid(17)

# Fixamos todas as células iniciais adicionais como mortas.
for cell in initial_seq:
    boundary_seq[cell] = False

# Ordem de atualizações do contraexemplo encontrado pelo Z3.
witness_order = [
    (2, 2), (1, 2), (3, 2),
    (1, 3), (2, 3), (1, 1),
    (3, 3), (2, 1), (3, 1)
]

trajectory_seq = [state_seq.copy()]

for cell in witness_order:
    new_value = next_cell_value(state_seq, boundary_seq, *cell)

    if new_value == state_seq[cell]:
        raise AssertionError(f"A célula {cell} não muda de estado.")

    state_seq = state_seq.copy()
    state_seq[cell] = new_value
    trajectory_seq.append(state_seq.copy())

# ------------------------------------------------
# 2. Modo simultâneo
# ------------------------------------------------
normal_sim, initial_sim, final_sim, boundary_sim, state_sim = create_grid(17)

for cell in initial_sim:
    boundary_sim[cell] = False

# Configuração inicial que originou o contraexemplo simultâneo.
live_boundary_cells = {
    (0, 16), (0, 13), (7, 0), (8, 0),
    (0, 2), (0, 11), (0, 17), (2, 0), (13, 0)
}

for cell in live_boundary_cells:
    boundary_sim[cell] = True

trajectory_sim = [state_sim.copy()]

for _ in range(3):
    state_sim = simultaneous_step(state_sim, boundary_sim)
    trajectory_sim.append(state_sim.copy())

# ------------------------------------------------
# 3. Verificação da propriedade
# ------------------------------------------------
values_seq = [int(state[target]) for state in trajectory_seq]
values_sim = [int(state[target]) for state in trajectory_sim]

return {
    "Célula analisada": target,
    "Sequencial - valores ao longo da execução": values_seq,
    "Sequencial - permaneceu sempre morta": all(v == 0 for v in values_seq),
    "Sequencial - estado final é erro": all(not v for v in state_seq.values()),
    "Simultâneo - valores ao longo da execução": values_sim,
    "Simultâneo - permaneceu sempre morta": all(v == 0 for v in values_sim),
    "Simultâneo - estado final é erro": all(not v for v in state_sim.values())
}
```

```
    demonstrate_permanently_dead_cell()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Procuramos configurações estáveis não vazias nos dois modos, identificando células que permanecem
vivas.
    """)
    return


@app.cell
def _(create_grid, next_cell_value, sequential_successors, simultaneous_step):
    def find_permanently_alive_witnesses():
        # Todas as células adicionais começam mortas.
        size = 17

        # ================================================
        # 1. Modo sequencial
        # ================================================
        normal_seq, initial_seq, final_seq, boundary_seq, state_seq = create_grid(size)

        for cell in initial_seq:
            boundary_seq[cell] = False

        # Sequência de células a atualizar, encontrada por simulação.
        witness_order = [
            (2, 3), (2, 1), (4, 2), (2, 2),
            (4, 3), (1, 1), (2, 1), (4, 1),
            (1, 3), (1, 2), (5, 2), (3, 2),
            (2, 2), (4, 2)
        ]

        for cell in witness_order:
            new_value = next_cell_value(state_seq, boundary_seq, *cell)

            if new_value == state_seq[cell]:
                raise AssertionError(f"A célula {cell} não muda de estado.")

            state_seq = state_seq.copy()
            state_seq[cell] = new_value

        sequential_live = sorted(
            cell for cell, alive in state_seq.items() if alive
        )

        sequential_stable = (
            len(sequential_successors(state_seq, boundary_seq)) == 0
            and bool(sequential_live)
        )

        # ================================================
        # 2. Modo simultâneo
        # ================================================
        normal_sim, initial_sim, final_sim, boundary_sim, state_sim = create_grid(size)

        for cell in initial_sim:
            boundary_sim[cell] = False

        simultaneous_generations = 0

        for _ in range(100):
            next_state = simultaneous_step(state_sim, boundary_sim)
            simultaneous_generations += 1

            if next_state == state_sim:
                break
```

```
            state_sim = next_state
        else:
            raise RuntimeError("Não foi encontrado um estado estável.")

        simultaneous_live = sorted(
            cell for cell, alive in state_sim.items() if alive
        )

        simultaneous_stable = (
            simultaneous_step(state_sim, boundary_sim) == state_sim
            and bool(simultaneous_live)
        )

        return {
            "SEQUENCIAL": {
                "Transições executadas": len(witness_order),
                "Células vivas no estado final": sequential_live,
                "Número de células vivas": len(sequential_live),
                "Existe estado estável não vazio": sequential_stable,
                "Célula (2, 1) permanece viva": state_seq[(2, 1)]
            },
            "SIMULTÂNEO": {
                "Gerações executadas": simultaneous_generations - 1,
                "Células vivas no estado final": simultaneous_live,
                "Número de células vivas": len(simultaneous_live),
                "Existe estado estável não vazio": simultaneous_stable
            }
        }


    find_permanently_alive_witnesses()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Utilizamos Z3 para confirmar as propriedades dos estados estáveis encontrados, distinguindo as transições normais da permanência no estado terminal.
    """)
    return


@app.cell
def _(
    create_grid,
    create_symbolic_state,
    next_cell_value,
    sequential_trace_transition,
    simultaneous_step,
    transition_predicate,
    z3,
):
    def z3_stable_witness_certificate():
        size = 17

        # ================================================
        # 1. Construir a configuração estável sequencial
        # ================================================
        normal_s, initial_s, final_s, boundary_s, state_s = create_grid(size)

        for cell in initial_s:
            boundary_s[cell] = False

        sequence_s = [
            (2, 3), (2, 1), (4, 2), (2, 2),
            (4, 3), (1, 1), (2, 1), (4, 1),
            (1, 3), (1, 2), (5, 2), (3, 2),
            (2, 2), (4, 2)
        ]
```

```python
for cell in sequence_s:
    value = next_cell_value(state_s, boundary_s, *cell)
    state_s = state_s.copy()
    state_s[cell] = value

# ================================================
# 2. Construir a configuração estável simultânea
# ================================================
normal_m, initial_m, final_m, boundary_m, state_m = create_grid(size)

for cell in initial_m:
    boundary_m[cell] = False

generations_m = 0

for _ in range(100):
    next_state_m = simultaneous_step(state_m, boundary_m)
    generations_m += 1

    if next_state_m == state_m:
        break

    state_m = next_state_m
else:
    raise RuntimeError("Não foi encontrada uma configuração estável.")


cells_s = tuple(sorted(normal_s | initial_s))

X_s = create_symbolic_state("cert_seq_x", cells_s)
Y_s = create_symbolic_state("cert_seq_y", cells_s)

X_m = create_symbolic_state("cert_sim_x", cells_s)
Y_m = create_symbolic_state("cert_sim_y", cells_s)

def bind_state(solver, symbolic_state, concrete_state, boundary):
    for cell in cells_s:
        value = concrete_state[cell] if cell in concrete_state else boundary[cell]
        solver.add(symbolic_state[cell] == z3.BoolVal(value))


ordinary_solver = z3.Solver()

bind_state(ordinary_solver, X_s, state_s, boundary_s)
bind_state(ordinary_solver, Y_s, state_s, boundary_s)

ordinary_solver.add(
    transition_predicate(
        X_s, Y_s, normal_s, initial_s,
        mode="sequential",
        selector_prefix="cert_ordinary"
    )
)

ordinary_result = ordinary_solver.check()

# Agora permitimos a transição de stuttering.
stutter_solver = z3.Solver()

bind_state(stutter_solver, X_s, state_s, boundary_s)
bind_state(stutter_solver, Y_s, state_s, boundary_s)

stutter_solver.add(
    sequential_trace_transition(
        X_s, Y_s, normal_s, initial_s
    )
)

stutter_result = stutter_solver.check()
```

```
        simultaneous_solver = z3.Solver()

        bind_state(simultaneous_solver, X_m, state_m, boundary_m)
        bind_state(simultaneous_solver, Y_m, state_m, boundary_m)

        simultaneous_solver.add(
            transition_predicate(
                X_m, Y_m, normal_m, initial_m,
                mode="simultaneous",
                selector_prefix="cert_simultaneous"
            )
        )

        simultaneous_result = simultaneous_solver.check()

        return {
            "Sequencial": {
                "Células vivas": sum(state_s.values()),
                "Transição normal X -> X": str(ordinary_result),
                "Transição de stuttering X -> X": str(stutter_result)
            },
            "Simultâneo": {
                "Células vivas": sum(state_m.values()),
                "Gerações até à estabilidade": generations_m - 1,
                "Transição normal X -> X": str(simultaneous_result)
            }
        }


    z3_stable_witness_certificate()
    return


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""
    Validamos novamente as trajetórias que conduzem aos estados estáveis, verificando as transições executadas e os resultados obtidos.
    """)
    return


@app.cell
def _(create_grid, next_cell_value, sequential_successors, simultaneous_step):
    def validate_stable_alive_paths():
        size = 17

        # ================================================
        # 1. Validar o percurso sequencial
        # ================================================
        normal_seq, initial_seq, final_seq, boundary_seq, state_seq = create_grid(size)

        for cell in initial_seq:
            boundary_seq[cell] = False

        sequence = [
            (2, 3), (2, 1), (4, 2), (2, 2),
            (4, 3), (1, 1), (2, 1), (4, 1),
            (1, 3), (1, 2), (5, 2), (3, 2),
            (2, 2), (4, 2)
        ]

        sequential_checks = []

        for cell in sequence:
            expected = next_cell_value(state_seq, boundary_seq, *cell)
            changed = expected != state_seq[cell]
            sequential_checks.append(changed)
```

```
            if not changed:
                raise AssertionError(
                    f"A transição sequencial na célula {cell} não é válida."
                )

            state_seq = state_seq.copy()
            state_seq[cell] = expected

        sequential_live = sorted(
            cell for cell, alive in state_seq.items() if alive
        )

        sequential_deadlock = len(
            sequential_successors(state_seq, boundary_seq)
        ) == 0

        # ================================================
        # 2. Validar o percurso simultâneo
        # ================================================
        normal_sim, initial_sim, final_sim, boundary_sim, state_sim = create_grid(size)

        for cell in initial_sim:
            boundary_sim[cell] = False

        simultaneous_counts = [
            sum(state_sim.values())
        ]

        simultaneous_steps = 0

        for _ in range(100):
            next_state = simultaneous_step(state_sim, boundary_sim)

            if next_state == state_sim:
                break

            state_sim = next_state
            simultaneous_steps += 1
            simultaneous_counts.append(sum(state_sim.values()))
        else:
            raise RuntimeError("Não foi encontrado um ponto fixo.")

        simultaneous_fixed_point = (
            simultaneous_step(state_sim, boundary_sim) == state_sim
        )

        return {
            "Sequencial": {
                "Número de transições": len(sequence),
                "Todas as transições são válidas": all(sequential_checks),
                "Número de células vivas no final": len(sequential_live),
                "Células vivas no final": sequential_live,
                "Estado bloqueado": sequential_deadlock
            },
            "Simultâneo": {
                "Número de gerações até ao ponto fixo": simultaneous_steps,
                "Células vivas por geração": simultaneous_counts,
                "Número de células vivas no final": sum(state_sim.values()),
                "Ponto fixo confirmado": simultaneous_fixed_point
            }
        }


    validate_stable_alive_paths()
    return


@app.cell(hide_code=True)
def _(mo):
```

```
    mo.md(r"""
    #Conclusão
    A implementação e verificação do Jogo da Vida permitiram encontrar contraexemplos para ambas as propriedades estudadas. Na modalidade sequencial, foi possível atingir o estado de erro após 9 transições, mesmo com todas as células adicionais inicialmente mortas. Na modalidade simultânea, esse estado foi alcançado após 3 gerações, numa configuração inicial admissível das células adicionais.
    Verificou-se também que uma célula normal pode permanecer sempre morta em ambas as modalidades. Quanto à permanência no estado vivo, foi encontrada, na modalidade sequencial, uma configuração terminal com 7 células vivas, e, na modalidade simultânea, uma configuração estável com 7 células vivas, atingida após 4 gerações. Os resultados confirmam, assim, que nenhuma das duas propriedades propostas se verifica em todas as trajetórias do sistema, ilustrando a utilidade conjunta da simulação em Python e da verificação simbólica com Z3.
    """)
    return


if __name__ == "__main__":
    app.run()
```